

03_info-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ info.py
```

El archivo `info.py` se encuentra dentro de la carpeta de rutas del backend:

```
backend/app/api/routes/
```

Este archivo define un endpoint informativo de la API. Su función es devolver metadatos básicos del backend, como el nombre del servicio, la versión de la aplicación, el identificador del commit, la fecha de construcción y la hora UTC actual.

Este router se importa y registra en `backend/app/main.py` mediante estas líneas:

```
from app.api.routes.info import router as info_router
```

```
app.include_router(info_router)
```

Gracias a esto, el endpoint definido en `info.py` queda incorporado a la aplicación FastAPI principal.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/app/api/routes/info.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/app/api/routes/info.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```

from __future__ import annotations

import os
from datetime import datetime, timezone

from fastapi import APIRouter

router = APIRouter(prefix="/info", tags=["info"])

@router.get("")
def info():
    return {
        "name": "siem-backend",
        "version": os.getenv("APP_VERSION", "0.1.0"),
        "git_sha": os.getenv("GIT_SHA", "unknown"),
        "build_time": os.getenv("BUILD_TIME", "unknown"),
        "utc_now": datetime.now(timezone.utc).isoformat(),
    }

```

4 Función general del archivo

El archivo `info.py` define un endpoint informativo del backend.

La ruta expuesta es:

```
GET /info
```

Cuando se llama a este endpoint, la API devuelve un objeto JSON con información básica del servicio:

```

{
  "name": "siem-backend",
  "version": "0.1.0",
  "git_sha": "unknown",
  "build_time": "unknown",
  "utc_now": "2026-05-26T..."
}

```

Este endpoint sirve para consultar información del backend sin acceder directamente al código ni a los contenedores.

En este proyecto, tiene relación directa con las variables de entorno definidas en Docker Compose:

```

APP_VERSION
GIT_SHA
BUILD_TIME

```

Estas variables se pasan al contenedor `siem-api` desde `docker/compose.yml`.

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
from __future__ import annotations
```

Importación futura para el manejo moderno de anotaciones.

```

import os
from datetime import datetime, timezone

```

Importaciones estándar de Python.

```
from fastapi import APIRouter
```

Importación de FastAPI para crear el router.

```
router = APIRouter(prefix="/info", tags=["info"])
```

Creación del router con prefijo `/info`.

```
@router.get("")
def info():
    return {...}
```

Definición del endpoint que devuelve la información del backend.

Visualmente:

```
info.py
├─ Importación futura
├─ Importaciones estándar de Python
├─ Importación de APIRouter
├─ Creación del router /info
└─ Endpoint GET /info
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa un comportamiento futuro de Python relacionado con las anotaciones de tipos.

Desglose:

```
from __future__
```

`__future__` es un módulo especial de Python que permite activar características modernas del lenguaje.

```
import annotations
```

Activa el comportamiento moderno de las anotaciones.

En este archivo concreto no se usan anotaciones de tipos en la función `info`, por lo que esta línea no tiene un efecto visible directo. Aun así, mantiene coherencia con otros archivos del proyecto que sí usan anotaciones, como `health.py`.

Importación del módulo `os`

```
import os
```

Esta línea importa el módulo estándar `os` de Python.

El módulo `os` permite interactuar con funcionalidades del sistema operativo y del entorno de ejecución.

En este archivo se utiliza para leer variables de entorno:

```
os.getenv("APP_VERSION", "0.1.0")
```

Las variables de entorno permiten configurar la aplicación desde fuera del código. En este proyecto, algunas se definen en `docker/compose.yml` y en los archivos `.env`.

Importación de `datetime` y `timezone`

```
from datetime import datetime, timezone
```

Esta línea importa dos elementos desde el módulo estándar `datetime` :

```
datetime  
timezone
```

`datetime`

`datetime` permite trabajar con fechas y horas.

En este archivo se usa aquí:

```
datetime.now(timezone.utc)
```

Sirve para obtener la fecha y hora actual.

`timezone`

`timezone` permite especificar una zona horaria.

En este archivo se usa:

```
timezone.utc
```

Esto indica que la hora generada debe estar en UTC.

UTC significa:

```
Coordinated Universal Time
```

En sistemas backend es habitual usar UTC porque evita problemas con cambios horarios, zonas locales o diferencias entre entornos.

Importación de `APIRouter`

```
from fastapi import APIRouter
```

Esta línea importa `APIRouter` desde FastAPI.

`APIRouter` permite definir un conjunto de rutas separado de la aplicación principal.

En este archivo se usa aquí:

```
router = APIRouter(prefix="/info", tags=["info"])
```

Después, este router se registra en `main.py` :

```
app.include_router(info_router)
```

Esto permite mantener las rutas informativas separadas del resto de endpoints.

Creación del router

```
router = APIRouter(prefix="/info", tags=["info"])
```

Esta línea crea un router de FastAPI.

Desglose:

```
router
```

Nombre de la variable donde se guarda el router.

Es importante que se llame `router`, porque `main.py` lo importa así:

```
from app.api.routes.info import router as info_router
```

```
=
```

Operador de asignación.

```
APIRouter(...)
```

Crea una instancia de router.

Parámetro `prefix`

```
prefix="/info"
```

Define el prefijo común de las rutas incluidas en este router.

Esto significa que todos los endpoints definidos en este archivo empezarán por:

```
/info
```

Como el decorador del endpoint usa una cadena vacía:

```
@router.get("")
```

la ruta final será:

```
GET /info
```

Parámetro `tags`

```
tags=["info"]
```

Define la etiqueta con la que aparecerá el endpoint en la documentación automática de FastAPI.

En Swagger, el endpoint aparecerá agrupado bajo la sección:

```
info
```

La documentación automática se puede consultar normalmente en:

```
http://localhost:8000/docs
```

Separación visual antes del endpoint

La línea en blanco entre la creación del router y el decorador no afecta al funcionamiento.

Sirve para separar visualmente la configuración del router de la definición del endpoint.

Decorador del endpoint

```
@router.get("")
```

Esta línea registra una ruta HTTP `GET` .

Desglose:

```
@
```

Indica que se está usando un decorador.

```
router
```

Es el router creado anteriormente.

```
.get
```

Indica que el endpoint responderá al método HTTP `GET` .

```
("")
```

Define la ruta relativa dentro del router.

Como el router tiene el prefijo:

```
prefix="/info"
```

y la ruta relativa está vacía, la ruta final será:

```
GET /info
```

Cuando llegue una petición a `/info` , FastAPI ejecutará la función situada justo debajo.

Definición de la función `info`

```
def info():
```

Esta línea define la función que se ejecutará al llamar al endpoint `GET /info` .

Desglose:

```
def
```

Palabra clave de Python para definir una función.

```
info
```

Nombre de la función.

```
()
```

La función no recibe parámetros.

A diferencia de `health.py` , este endpoint no necesita una sesión de base de datos, porque no consulta PostgreSQL. Solo lee variables de entorno y genera una fecha actual.

```
:
```

Los dos puntos indican el inicio del bloque de código de la función.

Inicio del diccionario de respuesta

```
return {
```

Esta línea indica que la función devolverá un diccionario de Python.

FastAPI convierte automáticamente ese diccionario en una respuesta JSON.

Desglose:

```
return
```

Devuelve un valor desde la función.

```
{
```

Inicia un diccionario.

Un diccionario en Python contiene pares clave-valor:

```
"clave": "valor"
```

Campo `name`

```
"name": "siem-backend",
```

Esta línea añade el campo `name` a la respuesta.

Desglose:

```
"name"
```

Clave del diccionario.

```
:
```

Separador entre clave y valor.

```
"siem-backend"
```

Valor asociado.

Este campo identifica el nombre del servicio backend.

La coma final indica que el diccionario continúa con más campos.

Respuesta parcial:

```
{  
  "name": "siem-backend"  
}
```

Campo `version`

```
"version": os.getenv("APP_VERSION", "0.1.0"),
```

Esta línea añade el campo `version` a la respuesta.

El valor se obtiene mediante:

```
os.getenv("APP_VERSION", "0.1.0")
```

Desglose:

```
os
```

Módulo estándar importado previamente.

```
.getenv(...)
```

Función que lee variables de entorno.

```
"APP_VERSION"
```

Nombre de la variable de entorno que se quiere leer.

```
"0.1.0"
```

Valor por defecto si la variable no existe.

Esto significa:

```
Lee la variable APP_VERSION.  
Si no existe, usa 0.1.0.
```

En Docker Compose, esta variable se define en el servicio `api` :

```
APP_VERSION: ${APP_VERSION:-0.1.0}
```

Por tanto, el endpoint `/info` puede mostrar la versión configurada desde el entorno.

Campo `git_sha`

```
"git_sha": os.getenv("GIT_SHA", "unknown"),
```

Esta línea añade el campo `git_sha` a la respuesta.

El valor se obtiene desde la variable de entorno:

```
GIT_SHA
```

Si no existe, se usa:

```
unknown
```

Desglose:

```
"git_sha"
```

Clave del diccionario.

```
os.getenv("GIT_SHA", "unknown")
```

Lee la variable de entorno `GIT_SHA` .

`GIT_SHA` suele representar el identificador de un commit de Git.

Esto permite saber qué versión exacta del código se está ejecutando, especialmente en despliegues o entornos donde se automatiza la construcción.

En este proyecto, si no se establece un valor real, se mantiene como:


```
unknown
```

Campo `build_time`

```
"build_time": os.getenv("BUILD_TIME", "unknown"),
```

Esta línea añade el campo `build_time` a la respuesta.

El valor se obtiene desde la variable de entorno:

```
BUILD_TIME
```

Si no existe, se usa:

```
unknown
```

Este campo puede indicar la fecha u hora en la que se construyó o desplegó la aplicación.

En este MVP, sirve como campo informativo y de trazabilidad.

Campo `utc_now`

```
"utc_now": datetime.now(timezone.utc).isoformat(),
```

Esta línea añade el campo `utc_now` a la respuesta.

El valor se genera dinámicamente cada vez que se llama al endpoint.

Desglose:

```
datetime
```

Clase importada desde el módulo `datetime`.

```
.now(...)
```

Método que obtiene la fecha y hora actual.

```
timezone.utc
```

Indica que la hora debe generarse en UTC.

```
.isoformat()
```

Convierte el objeto de fecha y hora a una cadena en formato ISO 8601.

El resultado tendrá un aspecto parecido a:

```
2026-05-26T15:42:10.123456+00:00
```

Este campo permite comprobar cuándo ha respondido el backend y evita depender de la hora local del navegador o del usuario.

Cierre del diccionario

```
}
```

Esta línea cierra el diccionario que devuelve la función.

FastAPI transformará automáticamente el diccionario en una respuesta HTTP en formato JSON.

Resultado final del archivo

Este archivo expone un endpoint:

```
GET /info
```

Su comportamiento es:

1. FastAPI recibe una petición GET /info.
2. Ejecuta la función info().
3. Lee variables de entorno mediante os.getenv.
4. Genera la hora UTC actual.
5. Devuelve un JSON con información del backend.

Respuesta esperada:

```
{
  "name": "siem-backend",
  "version": "0.1.0",
  "git_sha": "unknown",
  "build_time": "unknown",
  "utc_now": "2026-05-26T15:42:10.123456+00:00"
}
```

7 Relación con el flujo técnico del laboratorio

info.py no participa directamente en el flujo de ingesta de eventos, reglas o alertas. Su función es informativa y de diagnóstico.

La relación técnica sería:

```
Usuario / navegador / curl
    ↓
GET /info
    ↓
FastAPI ejecuta info()
    ↓
os.getenv lee variables de entorno
    ↓
datetime genera la hora UTC actual
    ↓
FastAPI devuelve JSON informativo
```

Este endpoint permite comprobar información del backend sin necesidad de acceder al contenedor ni revisar variables manualmente.

También conecta con la configuración de Docker Compose, porque algunas de las variables que devuelve se definen en el servicio `api`.

Relación con Docker:

```
docker/compose.yml
    ↓
environment:
  APP_VERSION
  GIT_SHA
  BUILD_TIME
    ↓
contenedor siem-api
    ↓
```

```
os.getenv(...)
  ↓
GET /info
```

8 Errores típicos o puntos importantes

Variables de entorno no definidas

Si `APP_VERSION`, `GIT_SHA` o `BUILD_TIME` no están definidas, el endpoint no falla.

En su lugar, usa valores por defecto:

```
APP_VERSION → 0.1.0
GIT_SHA      → unknown
BUILD_TIME   → unknown
```

Esto se debe al segundo argumento de `os.getenv`.

Ejemplo:

```
os.getenv("GIT_SHA", "unknown")
```

El endpoint no consulta la base de datos

A diferencia de `health.py`, este endpoint no usa `get_db` ni `Session`.

Esto significa que `/info` puede responder aunque PostgreSQL tenga algún problema, siempre que la API esté arrancada.

Por eso conviene diferenciar:

```
/info    → información del backend
/health  → estado de API + comprobación de base de datos
```

Importación del router en `main.py`

En `main.py`, este archivo se importa con:

```
from app.api.routes.info import router as info_router
```

Por tanto, `info.py` debe definir una variable llamada:

```
router
```

Si se cambiara el nombre, habría que modificar también `main.py`.

Uso de UTC

El campo:

```
"utc_now": datetime.now(timezone.utc).isoformat()
```

usa UTC en lugar de hora local.

Esto es recomendable en sistemas backend porque evita problemas con zonas horarias, cambios de hora o diferencias entre entornos.

Valor dinámico de `utc_now`

Cada llamada a `/info` genera un valor distinto para `utc_now`.

Esto confirma que la respuesta se está generando en tiempo real y no es un JSON estático.

9 Comandos útiles relacionados

Comprobar el endpoint desde el host:

```
curl http://localhost:8000/info
```

Respuesta esperada aproximada:

```
{
  "name": "siem-backend",
  "version": "0.1.0",
  "git_sha": "unknown",
  "build_time": "unknown",
  "utc_now": "2026-05-26T15:42:10.123456+00:00"
}
```

Comprobar el endpoint desde navegador:

```
http://localhost:8000/info
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Comprobar variables de entorno dentro del contenedor:

```
docker exec -it siem-api env | grep -E "APP_VERSION|GIT_SHA|BUILD_TIME"
```

Comprobar importación del router:

```
docker exec -it siem-api python -c "from app.api.routes.info import router; print(router)"
```

Comprobar importación de la app completa:

```
docker exec -it siem-api python -c "from app.main import app; print(app.title)"
```

Comprobar la documentación Swagger:

```
http://localhost:8000/docs
```